# Unix Lesson 7: Unix Power Tools: GREP, AWK, and SED

## Unix Power Tools

In addition to functions like `cut`, `sort`, `uniq` , the following tools extend the functionality of Unix command line immensely. They are powerful tools in their own right, however, when linked together through pipes they can perform task in half the time that modern scripting languages can do the same task in.

These tools are most typically suited for simple file cleaning, filtering, manipulation, and analysis. They aren't suited for complex statistical analysis or plotting.

We're going to cover 3 tools in this lesson.

| Command | Description |
|---|---|
| grep <pattern> <file(s)> | Search files for matching patter |
| awk | Scripting language to manipulate files and generate reports. There are many one line examples that can help do some powerful file manipulation. |
| sed | Stream Editor - performs search/replace, text manipulation, and editing on a stream (STDIN) |

## 7.1 `grep`: searching files for text patterns

The tools was developed to search for matches according to *regular expressions* in large text datasets. A *regular expression* is an expression that defines a pattern of a sequence of characters in text. See https://en.wikipedia.org/wiki/Regular_expression. We could spend a couple weeks on regular expression, however, we're just going to focus on using simple string matches during this course. If you are interested in exploring regulare expressions in more depth, please see https://regexr.com/ or https://medium.com/@nqabell89/4-helpful-websites-for-learning-to-use-regular-expressions-regex-d8b291f232ac

Usage: `grep 'pattern'`

To start off, say we're interested in identifying lines with the word: **ablative**

```
$ grep 'ablative' D43_Unix/Books/Latin_for_Beginners.txt
«49.» When the nominative singular ends in «-a», the ablative singular
```

```
ends in «-ā» and the ablative plural in «-īs».
...
```

## Exercise 1: How many female penguins were sampled in `D43_Unix/Data/penguins.tsv`.

Hint: Don't forget to use pipes `|` to pass the output to a program to count the output.

```
$ cd D43_Unix/Data

$ less penguins.tsv
rowid   species island  bill_length_mm  bill_depth_mm   flipper_length_mm
body_mass_g  sex     year
1       Adelie  Torgersen       39.1    18.7    181     3750    male    2007
2       Adelie  Torgersen       39.5    17.4    186     3800    female  2007

$ grep 'female' penguins.tsv | less

$ grep 'female' penguins.tsv | wc -l
```

## Exercise 2: How many samples are missing `sex` in `D43_Unix/Data/penguins.tsv` ?

```
$ less penguins.tsv

$ grep 'NA' penguins.tsv

$ grep 'NA' penguins.tsv | wc -l
```

To negate a pattern or rather find lines that don't have a certain pattern, use the `-v` option

```
$ grep 'ablative' D43_Unix/Books/Latin_for_Beginners.txt | grep -v
'singular'
ends in «-ā» and the ablative plural in «-īs».
...
```

---

### `zgrep`: Handles searching gzipped files

Similar to `cat` or `less`, `grep` has a version designed to deal directly with gzipped (compressed files)

---

## End Tuesday 8th April 2025

## Exercise 3: Find the number of entries in the file: `D43_Unix/Reference/Gencode/gencode.v47.primary_assembly.basic.annotation.gff3.gz` that are: 1) Type: `gene` 2) Attributes: `gene_type=protein_coding`

**GFF3 File Background:** The first line of a GFF3 file must be a comment that identifies the version, e.g.

```
##gff-version 3
```

Fields must be tab-separated. Also, all but the final field in each feature line must contain a value; "empty" columns should be denoted with a '.'

**seqid** - name of the chromosome or scaffold; chromosome names can be given with or without the 'chr' prefix. Important note: the seq ID must be one used within Ensembl, i.e. a standard chromosome name or an Ensembl identifier such as a scaffold ID, without any additional content such as species or assembly. See the example GFF output below.

**source** - name of the program that generated this feature, or the data source (database or project name)

**type** - type of feature. Must be a term or accession from the SOFA sequence ontology

**start** - Start position of the feature, with sequence numbering starting at 1.

**end** - End position of the feature, with sequence numbering starting at 1.

**score** - A floating point value.

**strand** - defined as + (forward) or - (reverse).

**phase** - One of '0', '1' or '2'. '0' indicates that the first base of the feature is the first base of a codon, '1' that the second base is the first base of a codon, and so on..

**attributes** - A semicolon-separated list of tag-value pairs, providing additional information about each feature. Some of these tags are predefined, e.g. ID, Name, Alias, Parent - see the GFF documentation for more details.

```
$ cd D43_Unix/Reference/Gencode

$ zless genocode.V47.primary_assembly.basic.annotation.gff3.gz
##gff-version 3
```

```
#description: evidence-based annotation of the human genome (GRCh38),
version 47 (Ensembl 113)
#provider: GENCODE
#contact: gencode-help@ebi.ac.uk
#format: gff3
#date: 2024-07-19
##sequence-region chr1 1 248956422
chr1    HAVANA  gene    11121   24894   .       +       .
ID=ENSG00000290825.2;gene_id=ENSG00000290825.2;gene_type=lncRNA;gene_name=DD
X11L16;level=2;tag=overlaps_pseudogene
chr1    HAVANA  transcript      11426   14409   .       +       .
ID=ENST00000832828.1;Parent=ENSG00000290825.2;gene_id=ENSG00000290825.2;tran
script_id=ENST00000832828.1;gene_type=lncRNA;gene_name=DDX11L16;transcript_t
ype=lncRNA;transcript_name=DDX11L16-264;level=2;tag=basic,TAGENE

$ zgrep 'gene' gencode.V47.primary_assembly.basic.annotation.gff3.gz | less

$ zgrep 'gene_type=protein_coding'
gencode.v47.primary_assembly.basic.annotation.gff3.gz | less

$ zgrep 'gene_type=protein_coding'
gencode.v47.primary_assembly.basic.annotation.gff3.gz | grep "ID=ENSG" |
less
```

I'd be really nice, if there were a tool which could filter based on a value of a particular column. `AWK` to the rescue.

---

## 7.2 `awk`

---

`awk` is a very powerful scripting language to match, modify, or process tabular text files. We're going to just focus on some basic one-line commands and show you how to interpret those commands. For more extended help, there are books and internet resources.

### What can we do with `awk`

#### 1. AWK Operations:

- Scans a file line by line
- Splits each input line into fields
- Compares input line/fields to pattern
- Performs action(s) on matched lines

**2. Useful For:**

- Transform data files
- Produce formatted reports

**3. Programming Constructs:**

- Format output lines
- Arithmetic and string operations
- Conditionals and loops

---

## 7.2.1 `awk` syntax

`awk options 'pattern {action}' input-file`

We're going to focus on the simple syntax as described above. There are 3 important parts:

1. **Options:**
   a. The default field separator is `space`.
   If you want to specify a different input separator, `-F <separator>` or see **Built-in Variables** Below
2. **Pattern**
   a. Which Lines should be ignored or selected. If not provided, all lines will be selected. Can issue multiple commands/assignments to variables, separated by `;`
3. **Action**
   a. What we want to do with the selected line - typically print all or a subset of fields from the table.

Let's start by going back to the `penguins` dataset.

```
$ awk '{print}' penguins.tsv

$ awk -F ',' '{print}' penguins.csv
```

### Print lines that match a given regular expression pattern

The regular expression pattern or word needs to be surrounded by `/<pattern>/`

To print out only the lines with `Gentoo` penguins

```
$ awk '/Gentoo/ {print}' penguins.tsv"
```

---

## Splitting Line into Fields

| Field Variable | Description |
|---|---|
| $0 | Complete Line |

| Field Variable | Description |
|---|---|
| $1 | First Columns |
| $2 | Second Column |
| $<n> | nth column |

**To print out the species and sex column for only the `Gentoo` penguins**

```
$ awk '/Gentoo/ {print $2,$8}' penguins.tsv
```

Note: When printing multiple outputs you can separate them with a comma and the output separator will be used.

---

## Conditioning on Field Value

We can use a single condition, such as matching, or testing for equality of a value in a given field. We can then combine those using **Logical Operators**.

**Relational Operators**

| Operator | Description |
|---|---|
| == | Equals |
| != | Not equal to |
| >= | Greater than or equal to |
| > | Greater than |
| <= | Less than or equal to |
| < | Less than |
| ~ | Matches string to regular expression |
| !~ | Doesn't match string to regular expression |

---

### Logical Operators

| Operator | Description |
|---|---|
| && | AND |
| \| | OR |
| ! | NOT |

**To get only the individuals with sex == `male`**

Note: If we are testing for equality and supplying a string, we need to surround he string with *double quotes*.

```
$ awk '$8=="male" {print}' penguins.tsv
```

**To get only the male Gentoo Biscoe penguins**

```
$awk '$2=="Gentoo" && $3=="Biscoe" && $8=="male" {print}' penguins.tsv
```

---

## Exercise 4: Find all the penguin samples that have a missing (NA) value for sex in `penguins.tsv` and save those samples to `Missing_sex.txt`

---

```
$ head penguins.tsv
rowid   species island  bill_length_mm  bill_depth_mm   flipper_length_mm
body_mass_g  sex     year
1       Adelie  Torgersen       39.1    18.7    181     3750    male    2007
2       Adelie  Torgersen       39.5    17.4    186     3800    female  2007

$ cut -f8 penguins.tsv | sort | uniq
NA
female
male
sex

$ awk '$8=="NA" {print}' penguins.tsv | less

$ awk '$8=="NA" {print}' penguins.tsv > Missing_Sex.txt
```

---

### The special case of `tab`, `newline`, `carriage return` characters

When programming, we often want to specify a `tab`, `newline`, or `carriage return` character. We can represent these as:

| Character | Unix Representation |
|---|---|
| tab | \t |
| newline | \n |
| carriage return | \r |

**To print out the species and sex column for only the `Gentoo` penguins**

```
$ awk '/Gentoo/ {print $2"\t"$8}' penguins.tsv
```

Note: When printing multiple outputs you can separate them with a comma and the output separator will be used, which by default is a SPACE.

```
$ awk '/Gentoo/ {print $2,$8}' penguins.tsv
```

---

## Exercise 5: To print out the penguins species, body_mass_g, and sex column separated by tabs for only the `Gentoo` species with a body_mass_g greater than 4000gram

---

```
$ head penguins.txt

$ awk '$2=="Gentoo" {print}' penguins.tsv | less

$ awk '$2=="Gentoo" && $7>4000 {print}' penguins.tsv | less

$ awk '$2=="Gentoo" && $7>4000 {print $2,$7,$8}' penguins.tsv | less

$ awk '$2=="Gentoo" && $7>4000 {print $2"\t"$7"\t"$8}' penguins.tsv | less
```

---

### Modify the Output Values

AWK works great to modify numerical values. It allows you to quickly add a constant to values or calculate new values.

#### Mathematical Operators

| Operator | Description |
|---|---|
| + | Add |
| - | Subtract |
| / | Divide |
| * | Multiplication |
| % | Modulus (Remainder of integer division) |

## Exercise 6: Calculate the body_mass_g / bill_length_mm for all the Adelie penguins.

---

```
$ less penguins.tsv
```

```
$ awk '$4>"NA" && $7!="NA" {print}' penguins.tsv | less
rowid   species island  bill_length_mm  bill_depth_mm   flipper_length_mm
body_mass_g  sex     year
1       Adelie  Torgersen       39.1    18.7    181     3750    male    2007
2       Adelie  Torgersen       39.5    17.4    186     3800    female  2007

$ awk '$4!="NA" && $7!="NA" {print $7/$4 }' penguins.tsv | less
awk: cmd. line:1: (FILENAME=penguins.tsv FNR=1) fatal: division by zero
attempted

# What is the result of dividing two words:  "body_mass_g"/"bill_length_mm"?
# It's undefined as strings don't have numerical values -
# by default they are interpreted as value zero numerically.

$ awk '$1!="rowid" {print}' penguins.tsv | awk '$4!="NA" && $7!="NA" {print
$7/$4 }' | less
```

---

## AWK Built-in Variables

AWK is a very powerful tool and is actually a whole scripting language. It has some additional built-in variables that allow you to set the separators between fields or records either from STDIN or for STDOUT. The semi-colon `;` is a separator for different commands

| AWK Built-in Variables | Description |
|---|---|
| FS | Input Field Separator (Default: Space/Tab) |
| OFS | Output Field Separator (Default: Space) |
| RS | Input Record Separator (Line Separator) (Default: \n) |
| ORS | Output Records Separator (Default: \n) |
| NR | Current # of records processed |
| NF | Number of Fields in the record |

## Redo Exercise 3 with Grep and Awk:

```
$ zgrep 'gene_type=protein_coding'
gencode.v47.primary_assembly.basic.annotation.gff3.gz | awk '$3=="gene"
{print}' | less

$ zgrep 'gene_type=protein_coding'
gencode.v47.primary_assembly.basic.annotation.gff3.gz | awk
'OFS="\t";$3=="gene" {print}' | less
```

How did the outputs differ between the **original Exercise 3** and when we used AWK?

In the previous **Redo Exercise 3,** we used AWK to parse the fields and only return records where the 3rd column was **gene**. However, there was a difference between the separators of the input (TAB) and the STDOUT output (SPACE). To ensure that the output field separator was a TAB character, we assigned `OFS="\t";` The semi-colon `;` is a separator for different commands in AWK (ie. `OFS="\t"` and `$3=="gene"` need to be separated by `;`)

---

## 7.3 SED (Stream Editor)

SED allows you to quickly search and replace text.

**Basic Syntax:** `sed <options> '<command>'`

### 7.3.1 Search and Replace `'s/<pattern>/<replace>/<occurance option>'`

The pattern can be a simple string or a regular expression. When a match is found, it'll replace a the matched patterns with the replacement value. By default, SED replaces only the first occurrance.

Occurrance Options

| Option | Description |
|---|---|
| None | Only replaces 1st match per line |
| `g` | Replaces all (globally) |
| `<n>` | Replaces the nth occurance |
| `<n>g` | Replace nth occurance and above |

## Exercise 6: Replace the all the `tab` characters in the first line of `penguins.tsv` with newline `\n` characters.

```
$ head -n1 penguins.tsv
rowid   species island  bill_length_mm  bill_depth_mm   flipper_length_mm
body_mass_g  sex     year

$ head -n1 penguins.tsv | sed 's/\t/\n/g'
```

## Exercise 7: Use `head`, `sed`, and `cat` to determine what the columns number are for the columns in `penguins.tsv`

```
$ head -n1 penguins.tsv

$ head -n1 penguins.tsv | sed 's/\t/\n/g'

$ man cat

$ head -n1 penguins.tsv | sed 's/\t/\n/g' | cat -n
```

### 7.3.1 Delete Given Line `1d`

SED allows you to specify a given line, range of lines, or pattern of lines to delete.

**Delete Command**

| Option | Description |
|---|---|
| `1d` | Delete the first line |
| `2d` | Delete the second line |
| `<n>d` | Delete the nth line |
| `$d` | Delete the last line |
| `<m>,<n>d` | Delete mth to nth line |
| `<n>,$d` | Delete nth to last line |
| `/<pattern>/d` | Delete lines that match a given pattern |

---

## Exercise 8: Delete the first line of `penguins.tsv`

```
$ less penguins.tsv
rowid   species island  bill_length_mm  bill_depth_mm   flipper_length_mm
body_mass_g  sex     year
1       Adelie  Torgersen       39.1    18.7    181     3750    male    2007
2       Adelie  Torgersen       39.5    17.4    186     3800    female  2007

$ sed '1d' penguins.tsv | less
```

## Exercise 9: Delete the header lines (ie. lines that start with #) from D43_Unix/Reference/Clinvar/Clinvar_20241111.vcf.gz

```
$ zless Clinvar_20241111.vcf.gz
##fileformat=VCFv4.1
```

```
##fileDate=2024-11-11
##source=ClinVar
##reference=GRCh38

$ zcat Clinvar_20241111.vcf.gz | sed '/#/d' | less
```

---

## Week Assignment: Convert a VCF (D43_Unix/Reference/Clinvar/Clinvar_20241111.vcf.gz) to a BED File called Clinvar.bed

### VCF Format

A Variant Call File contains variant calls from sequencing.  Each entry is a position where a difference has been identified.

VCF files start with a header.  Each header line is prefixed with `##` and ends with a column header:

`#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT <SAMPLE(S)>`

If no sample level information is provided in the VCF, then FORMAT and SAMPLE columns are not included.

| Columns | Description |
|---|---|
| CHROM | Chromosome. An identifier from the reference genome |
| POS | Position: The reference position, with the 1st base having position 1 |
| ID | Identifier: Semicolon-separated list of unique identifiers where available |
| REF | Reference base(s): Each base must be one of A,C,G,T,N (case insensitive) |
| ALT | Alternate base(s): Comma separated list of alternate non-reference alleles |
| QUAL | Quality: Phred-scaled quality score for the assertion made in ALT. |
| FILTER | Filter status: PASS if this position has passed all filters, i.e., a call is made at this position |
| INFO | additional information: (String, no whitespace, semicolons, or equals-signs permitted; commas are permitted only as delimiters for lists of values) |

**VCF Specifications:** https://samtools.github.io/hts-specs/VCFv4.2.pdf

### BED Format

BED (Browser Extensible Data) format provides a flexible way to define the data lines that are displayed in an annotation track. BED lines have three required fields and nine additional optional fields. The number of fields per line must be consistent throughout any single set of data in an annotation track.

| Column | Description |
|---|---|
| chrom | The name of the chromosome (e.g. chr3, chrY, chr2_random) or scaffold (e.g. scaffold10671). Many assemblies also support several different chromosome aliases (e.g. '1' or 'NC_000001.11' in place of 'chr1'). |
| chromStart | The starting position of the feature in the chromosome or scaffold. The first base in a chromosome is numbered 0. |
| chromEnd | The ending position of the feature in the chromosome or scaffold. Position is not included in 0-base count or position is 1-base |

**BED Format Specifications:** https://genome.ucsc.edu/FAQ/FAQformat.html#format1

For this assignment, we're going to ignore the REF and ALT sequence length and just consider the CHROM and POS columns.

**Example:** If you were given a VCF with two lines and asked to convert the VCF to a BED file the input and output would look like:

**INPUT:** VCF

```
##fileformat=VCFv4.1
#CHROM  POS     ID      REF     ALT     QUAL    FILTER  INFO
1       69134   2205837 A       G       .       .       ALLELEID=2193183;
1       69314   3205580 T       G       .       .       ALLELEID=3374047;
```

**OUTPUT:** BED

The BED file would look like:

```
1       69133   69134
1       69313   69314
```

---

## Additional Lesson Reference:

https://www-users.york.ac.uk/~mijp1/teaching/2nd_year_Comp_Lab/guides/grep_awk_sed.pdf
https://missing.cs.ucdavis.edu/modules/scripting/text-processing/
https://seneca-ictoer.github.io/ULI101/A-Tutorials/tutorial10
https://www.geeksforgeeks.org/awk-command-unixlinux-examples/
https://www.geeksforgeeks.org/grep-command-in-unixlinux/
https://www.geeksforgeeks.org/sed-command-in-linux-unix-with-examples/

---

**Extra Credit:** [https://www.geeksforgeeks.org/tr-command-in-unix-linux-with-examples/](https://www.geeksforgeeks.org/tr-command-in-unix-linux-with-examples/)

---